In [1]:
```
# install required packages
%pip install -q -U "transformers>=4.50.0" accelerate huggingface_hub safetensors sentencepiece
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/12.6 MB ? eta -:--:--
   ━╸━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.6/12.6 MB 16.5 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/12.6 MB 75.5 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╸ 12.6/12.6 MB 233.1 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.6/12.6 MB 126.4 MB/s eta 0:00:00
```

In [2]:
```
# Deep Learning Functionality
import torch
from transformers import AutoProcessor, AutoModelForImageTextToText

# Image Processing
from PIL import Image

# Data Handling And Analysis
import pandas as pd

# Utilities
import gc
import json
import random
import shutil
import tarfile
import time
from pathlib import Path

# Hugging Face Authentication
from huggingface_hub import login

# Progress Monitoring
from tqdm.auto import tqdm
```

In [3]:
```
# display complete DataFrame columns
pd.set_option("display.max_colwidth", None)
```

In [5]:
```
# authenticate with Hugging Face
try:
    from google.colab import userdata
```

```
    hf_token = userdata.get("HF_TOKEN")
except Exception:
    hf_token = None

if hf_token:
    login(token=hf_token, add_to_git_credential=False)
else:
    login(add_to_git_credential=False)
```



To log in, open this URL and enter the code:

**https://hf.co/oauth/device**

## MT09-UR4J

*Waiting for authorization...*

Login successful. Logged in as **hodsong** (token: `oauth-hodsong`).

This token will be refreshed automatically when it expires.

In [6]:
```
# project folder
project_folder = Path("/content/drive/MyDrive/OCOM5300M_VLM_Benchmark")

# test captions filename
test_captions_filename = project_folder / "test_captions.csv"

# test image folder and archive
test_image_folder = project_folder / "test_images"
test_image_archive = project_folder / "rocov2_test_images.tar"

# select archive or extracted test images
use_image_archive = False
```

```
# output folder
output_folder = project_folder / "outputs"
output_folder.mkdir(parents=True, exist_ok=True)
```

In [7]:
```
# generated captions filename
vlm_generated_captions_filename = output_folder / "VLM_generated_captions.csv"

# checkpoint and diagnostics filenames
checkpoint_filename = output_folder / "VLM_generated_captions_checkpoint.csv"
diagnostics_filename = output_folder / "VLM_generation_diagnostics.csv"
errors_filename = output_folder / "VLM_generation_errors.csv"
configuration_filename = output_folder / "VLM_generation_configuration.json"
```

In [8]:
```
# set random seed for reproducibility
seed = 42

random.seed(seed)
torch.manual_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

In [9]:
```
# select GPU
if not torch.cuda.is_available():
    raise RuntimeError("Enable a GPU in Colab: Runtime > Change runtime type > GPU")

device = torch.device("cuda:0")
gpu_name = torch.cuda.get_device_name(0)
gpu_memory_gb = torch.cuda.get_device_properties(0).total_memory / (1024 ** 3)

# select model precision
model_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16

print(f"GPU: {gpu_name}")
print(f"GPU memory: {gpu_memory_gb:.2f} GB")
print(f"Model precision: {model_dtype}")
```

```
GPU: NVIDIA A100-SXM4-40GB
GPU memory: 39.49 GB
Model precision: torch.bfloat16
```

In [10]:
```python
# confirm input files exist
if not test_captions_filename.is_file():
    raise FileNotFoundError(f"Test captions file not found: {test_captions_filename}")

if use_image_archive and not test_image_archive.is_file():
    raise FileNotFoundError(f"Test image archive not found: {test_image_archive}")

if not use_image_archive and not test_image_folder.is_dir():
    raise FileNotFoundError(f"Test image folder not found: {test_image_folder}")
```

In [11]:
```python
# extract test images if using the archive
if use_image_archive:
    local_folder = Path("/content/roco_medgemma")
    image_folder = local_folder / "test_images"
    extraction_marker = local_folder / "extraction_complete.txt"
    local_folder.mkdir(parents=True, exist_ok=True)

    if not extraction_marker.exists():
        if image_folder.exists():
            shutil.rmtree(image_folder)
        image_folder.mkdir(parents=True, exist_ok=True)

        local_archive = local_folder / test_image_archive.name
        shutil.copy2(test_image_archive, local_archive)

        # extract the archive safely
        with tarfile.open(local_archive, "r:*") as archive:
            archive.extractall(image_folder, filter="data")

        extraction_marker.write_text("complete", encoding="utf-8")
        local_archive.unlink(missing_ok=True)
else:
    image_folder = test_image_folder

print(f"Test image folder: {image_folder}")
```

```
Test image folder: /content/drive/MyDrive/OCOM5300M_VLM_Benchmark/test_images
```

In [12]:
```python
# load test image IDs without using reference captions
test_caption_data = pd.read_csv(test_captions_filename, dtype={"ID": str})
```

```
if "ID" not in test_caption_data.columns:
    raise KeyError("The test captions CSV must contain an ID column")

if test_caption_data["ID"].isna().any():
    raise ValueError("The test captions CSV contains missing image IDs")

test_image_ids = test_caption_data["ID"].str.strip().drop_duplicates().tolist()

if "" in test_image_ids:
    raise ValueError("The test captions CSV contains blank image IDs")

print(f"Unique ROCOv2 test images: {len(test_image_ids)}")
```

```
Unique ROCOv2 test images: 9927
```

In [13]:
```
# index test image files
image_extensions = {".jpg", ".jpeg", ".png"}
image_path_lookup = {}

for image_path in image_folder.rglob("*"):
    if image_path.is_file() and image_path.suffix.lower() in image_extensions:
        if image_path.stem in image_path_lookup:
            raise ValueError(f"Duplicate image filename ID: {image_path.stem}")
        image_path_lookup[image_path.stem] = image_path

# confirm test image coverage
missing_image_ids = [image_id for image_id in test_image_ids if image_id not in image_path_lookup]

print(f"Images available: {len(test_image_ids) - len(missing_image_ids)}")
print(f"Missing images: {len(missing_image_ids)}")

if missing_image_ids:
    raise FileNotFoundError(f"Missing test images, first 10 IDs: {missing_image_ids[:10]}")
```

```
Images available: 9927
Missing images: 0
```

In [14]:
```
# pretrained MedGemma model
model_name = "google/medgemma-1.5-4b-it"
```

In [15]:
```
# medical image captioning prompt
caption_prompt = (
```

```
    "Generate one concise radiology image caption describing only the visible image. "
    "Do not invent findings that are not visible. "
    "Return only the caption."
)
```

In [16]:
```
# caption generation settings
maximum_new_tokens = 96

generation_configuration = {
    "max_new_tokens": maximum_new_tokens,
    "do_sample": False,
    "num_beams": 1,
    "use_cache": True,
}

generation_configuration
```

Out[16]:
```
{'max_new_tokens': 96, 'do_sample': False, 'num_beams': 1, 'use_cache': True}
```

In [17]:
```
# Load MedGemma processor
processor = AutoProcessor.from_pretrained(model_name)
```

processor_config.json: 100% 70.0/70.0 [00:00<00:00, 9.30kB/s]

chat_template.jinja: 100% 1.53k/1.53k [00:00<00:00, 195kB/s]

preprocessor_config.json: 100% 570/570 [00:00<00:00, 75.8kB/s]

config.json: 100% 2.55k/2.55k [00:00<00:00, 309kB/s]

tokenizer_config.json: 100% 1.16M/1.16M [00:00<00:00, 53.7MB/s]

tokenizer.json: reconstructing file: 100% 33.4MB / 33.4MB, 3.30MB/s

tokenizer.json: downloading bytes: 8.82MB, 871kB/s

added_tokens.json: 100% 35.0/35.0 [00:00<00:00, 4.65kB/s]

special_tokens_map.json: 100% 662/662 [00:00<00:00, 89.2kB/s]

In [18]:
```python
# Load pretrained MedGemma model
medgemma = AutoModelForImageTextToText.from_pretrained(
    model_name,
    torch_dtype=model_dtype,
    device_map="auto",
    low_cpu_mem_usage=True,
)

# set evaluation mode
medgemma.eval()
model_device = next(medgemma.parameters()).device

print(f"Model loaded: {model_name}")
print(f"Model device: {model_device}")
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
```

model.safetensors.index.json: 100%  90.6k/90.6k [00:00<00:00, 10.9MB/s]

Download complete:  7.79GB, 1.32GB/s

Reconstruction complete: 100%  8.60GB / 8.60GB, 490MB/s

Fetching 2 files: 100%  2/2 [00:27<00:00, 27.01s/it]

Loading weights: 100%  883/883 [00:02<00:00, 980.96it/s]

generation_config.json: 100%  115/115 [00:00<00:00, 15.5kB/s]

```
Model loaded: google/medgemma-1.5-4b-it
Model device: cuda:0
```

In [19]:
```python
def clean_generated_caption(caption):
    """
    Remove additional whitespace and an optional caption prefix.

    Args:
        caption (str): decoded MedGemma response

    Returns:
        str: cleaned generated caption
    """
    try:
```

```
        caption = " ".join(str(caption).strip().split())

        if caption.lower().startswith("caption:"):
            caption = caption[len("caption:"):].strip()

        return caption

    except Exception as e:
        raise RuntimeError(f"Exception: {e}")
```

In [20]:

```
def generate_medgemma_caption(image_path):
    """
    Generate one caption for a ROCOv2 test image.

    Args:
        image_path (Path): path to the test image

    Returns:
        tuple: generated caption and inference diagnostics
    """
    try:
        # Load RGB image
        with Image.open(image_path) as image_file:
            image = image_file.convert("RGB")

        # construct image and text prompt
        messages = [{
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text", "text": caption_prompt},
            ],
        }]

        # prepare model inputs
        inputs = processor.apply_chat_template(
            messages,
            add_generation_prompt=True,
            tokenize=True,
            return_dict=True,
            return_tensors="pt",
```

```
        ).to(model_device, dtype=model_dtype)

        input_token_count = inputs["input_ids"].shape[-1]

        # generate caption
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()
        start_time = time.perf_counter()

        with torch.inference_mode():
            output_tokens = medgemma.generate(**inputs, **generation_configuration)

        torch.cuda.synchronize()
        generation_seconds = time.perf_counter() - start_time

        # decode generated tokens
        generated_tokens = output_tokens[0, input_token_count:]
        generated_caption = clean_generated_caption(
            processor.decode(generated_tokens, skip_special_tokens=True)
        )

        if not generated_caption:
            raise ValueError("MedGemma generated an empty caption")

        # record generation diagnostics
        diagnostics = {
            "Input Tokens": int(input_token_count),
            "Output Tokens": int(generated_tokens.shape[0]),
            "Generated Words": len(generated_caption.split()),
            "Generation Seconds": generation_seconds,
            "CUDA Peak Allocated GB": torch.cuda.max_memory_allocated() / (1024 ** 3),
        }

        del inputs, output_tokens, generated_tokens, image
        return generated_caption, diagnostics

    except Exception as e:
        raise RuntimeError(f"Exception: {e}")
```

In [21]:

```
# save model and prompt configuration
run_configuration = {
```

```
    "model_name": model_name,
    "caption_prompt": caption_prompt,
    "generation_configuration": generation_configuration,
    "seed": seed,
    "test_image_count": len(test_image_ids),
}

# verify previous run configuration
previous_predictions_exist = checkpoint_filename.exists() or vlm_generated_captions_filename.exists()

if previous_predictions_exist and configuration_filename.exists():
    previous_configuration = json.loads(configuration_filename.read_text(encoding="utf-8"))
    if previous_configuration != run_configuration:
        raise ValueError("Existing predictions use different model or prompt settings. Start with a new output
folder.")
elif previous_predictions_exist:
    raise ValueError("Existing predictions have no configuration file. Start with a new output folder.")

configuration_filename.write_text(json.dumps(run_configuration, indent=4), encoding="utf-8")
```

Out[21]:
```
419
```

In [22]:
```
# load previously generated captions
resume_filename = checkpoint_filename if checkpoint_filename.exists() else vlm_generated_captions_filename

if resume_filename.exists():
    previous_captions = pd.read_csv(resume_filename, dtype={"ID": str})
    if previous_captions["ID"].duplicated().any():
        raise ValueError("Saved predictions contain duplicate image IDs")
    previous_captions = previous_captions.dropna(subset=["ID", "generated_caption"])
    caption_records = dict(zip(previous_captions["ID"], previous_captions["generated_caption"]))
else:
    caption_records = {}

# load previous diagnostics and errors
if diagnostics_filename.exists():
    generation_diagnostics = pd.read_csv(diagnostics_filename, dtype={"ID": str}).set_index("ID").to_dict("index")
else:
    generation_diagnostics = {}

if errors_filename.exists():
```

```
    previous_errors = pd.read_csv(errors_filename, dtype={"ID": str})
    generation_errors = dict(zip(previous_errors["ID"], previous_errors["Error"]))
else:
    generation_errors = {}

# identify remaining test images
remaining_image_ids = [image_id for image_id in test_image_ids if image_id not in caption_records]

print(f"Captions already generated: {len(caption_records)}")
print(f"Images remaining: {len(remaining_image_ids)}")
```

```
Captions already generated: 0
Images remaining: 9927
```

In [23]:

```
def save_generation_progress():
    """
    Save the current captions, diagnostics and errors to Google Drive.

    Returns:
        None
    """
    # retain original test image order
    caption_data = pd.DataFrame(
        [(image_id, caption_records[image_id]) for image_id in test_image_ids if image_id in caption_records],
        columns=["ID", "generated_caption"],
    )
    caption_data.to_csv(checkpoint_filename, index=False)

    diagnostic_data = pd.DataFrame(
        [{"ID": image_id, **values} for image_id, values in generation_diagnostics.items()],
        columns=["ID", "Input Tokens", "Output Tokens", "Generated Words", "Generation Seconds", "CUDA Peak
Allocated GB"],
    )
    diagnostic_data.to_csv(diagnostics_filename, index=False)

    error_data = pd.DataFrame(
        [{"ID": image_id, "Error": error} for image_id, error in generation_errors.items()],
        columns=["ID", "Error"],
    )
    error_data.to_csv(errors_filename, index=False)
```

In [24]:
```python
# generate one test caption
run_single_image_test = True

if run_single_image_test:
    single_image_id = remaining_image_ids[0] if remaining_image_ids else test_image_ids[0]
    single_image_path = image_path_lookup[single_image_id]

    single_caption, single_diagnostics = generate_medgemma_caption(single_image_path)

    print(f"Image ID: {single_image_id}")
    print(f"Generated Caption: {single_caption}")
    print(single_diagnostics)
```

```
Image ID: ROCOv2_2023_test_000001
Generated Caption: Chest X-ray showing a large, rounded opacity in the right upper mediastinum, possibly representing
a mass or enlarged lymph node. The heart size appears enlarged. The lungs are clear.
{'Input Tokens': 294, 'Output Tokens': 38, 'Generated Words': 29, 'Generation Seconds': 4.846074211999621, 'CUDA Peak
Allocated GB': 8.12458610534668}
```

In [25]:
```python
# enable full test set generation when ready
run_full_test_set = False

# checkpoint frequency
checkpoint_every = 25
```

In [27]:
```python
run_full_test_set = True
```

In [28]:
```python
# generate remaining captions
if run_full_test_set:
    newly_generated = 0

    try:
        for image_id in tqdm(remaining_image_ids, desc="MedGemma captions", unit="image"):
            generation_succeeded = False

            try:
                generated_caption, diagnostics = generate_medgemma_caption(image_path_lookup[image_id])

                caption_records[image_id] = generated_caption
                generation_diagnostics[image_id] = diagnostics
                generation_errors.pop(image_id, None)
```

```
                newly_generated += 1
                generation_succeeded = True

            except Exception as e:
                generation_errors[image_id] = str(e)

            # save generation progress
            if generation_succeeded and newly_generated % checkpoint_every == 0:
                save_generation_progress()

                gc.collect()
                torch.cuda.empty_cache()

    finally:
        # save completed predictions
        save_generation_progress()

    print(f"Generated captions saved in checkpoint: {len(caption_records)}")
    print(f"Images with recorded errors: {len(generation_errors)}")
```

MedGemma captions: 100% 9927/9927 [5:44:23<00:00, 2.36s/image]

```
Generated captions saved in checkpoint: 9927
Images with recorded errors: 0
```

In [29]:

```
# construct generated captions DataFrame
vlm_generated_captions = pd.DataFrame(
    [(image_id, caption_records[image_id]) for image_id in test_image_ids if image_id in caption_records],
    columns=["ID", "generated_caption"],
)

# confirm prediction coverage
expected_image_count = len(test_image_ids)
generated_image_count = len(vlm_generated_captions)

print(f"Expected ROCOv2 test images: {expected_image_count}")
print(f"Generated captions: {generated_image_count}")
print(f"Remaining images: {expected_image_count - generated_image_count}")

vlm_generated_captions.head()
```

```
Expected ROCOv2 test images: 9927
Generated captions: 9927
Remaining images: 0
```

Out[29]:

| | ID | generated_caption |
|---|---|---|
| **0** | ROCOv2_2023_test_000001 | Chest X-ray showing a large, rounded opacity in the right upper mediastinum, possibly representing a mass or enlarged lymph node. The heart size appears enlarged. The lungs are clear. |
| **1** | ROCOv2_2023_test_000002 | Coronary artery CT angiography showing a partially visualized left coronary artery. |
| **2** | ROCOv2_2023_test_000003 | Angiogram of the lower extremities showing a catheter tip in the left common iliac artery. |
| **3** | ROCOv2_2023_test_000004 | Angiogram of the lower extremities. A catheter is seen within the left common iliac artery. The right common iliac artery is also visible. The left external iliac artery is visualized. The right external iliac artery is also visible. The left common femoral artery is visible. The right common femoral artery is also visible. The left profunda femoris artery is visible. The right profunda femoris artery is also visible. The left superficial femoral artery is visible. The right superficial |
| **4** | ROCOv2_2023_test_000005 | X-ray of the shoulder joint showing the glenohumeral joint. The image displays the humeral head and glenoid fossa. The angle between the humeral head and glenoid fossa is indicated. |

In [30]:

```python
# validate complete test set
valid_captions = (
    vlm_generated_captions["generated_caption"].notna().all()
    and vlm_generated_captions["generated_caption"].astype(str).str.strip().ne("").all()
)

complete_test_set = (
    generated_image_count == expected_image_count
    and vlm_generated_captions["ID"].is_unique
    and set(vlm_generated_captions["ID"]) == set(test_image_ids)
    and valid_captions
)

if complete_test_set:
    # save CSV for Performance Notebook
    vlm_generated_captions = vlm_generated_captions[["ID", "generated_caption"]]
    vlm_generated_captions.to_csv(vlm_generated_captions_filename, index=False)
    print(f"Final CSV saved: {vlm_generated_captions_filename}")
else:
```

```
    print("The full test set is not complete. Resume generation before exporting the final CSV.")
    print(f"Checkpoint: {checkpoint_filename}")
```

```
Final CSV saved: /content/drive/MyDrive/OCOM5300M_VLM_Benchmark/outputs/VLM_generated_captions.csv
```

In [31]:

```
# display generated caption statistics
if not vlm_generated_captions.empty:
    generated_caption_lengths = vlm_generated_captions["generated_caption"].astype(str).str.split().str.len()
    print(generated_caption_lengths.describe())
    print(f"Unique generated captions: {vlm_generated_captions['generated_caption'].nunique()}")
else:
    print("No captions generated yet")
```

```
count    9927.000000
mean       20.869648
std        16.763876
min         1.000000
25%        10.000000
50%        15.000000
75%        26.000000
max        91.000000
Name: generated_caption, dtype: float64
Unique generated captions: 7955
```


Google (2026) *MedGemma 1.5 4B model documentation*. Available At https://developers.google.com/health-ai-developer-foundations/medgemma/model-card

Hugging Face (2026) *google/medgemma-1.5-4b-it*. Available At https://huggingface.co/google/medgemma-1.5-4b-it

Hugging Face (2026) *Transformers Documentation*. Available At https://huggingface.co/docs/transformers/

Google (2026) *Google Colaboratory*. Available At https://colab.research.google.com/